CookMem Tutorial 1

Getting Started with CookMem

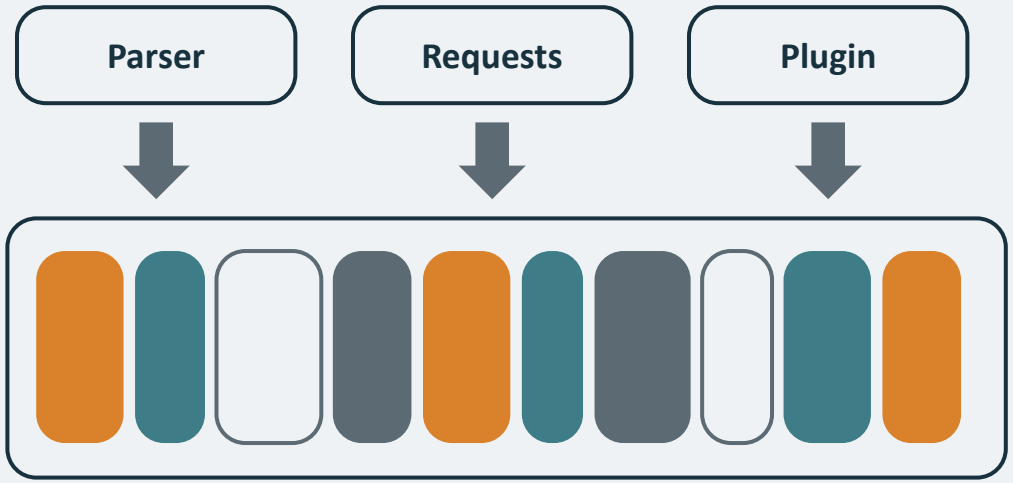
An overview of memory contexts in C++



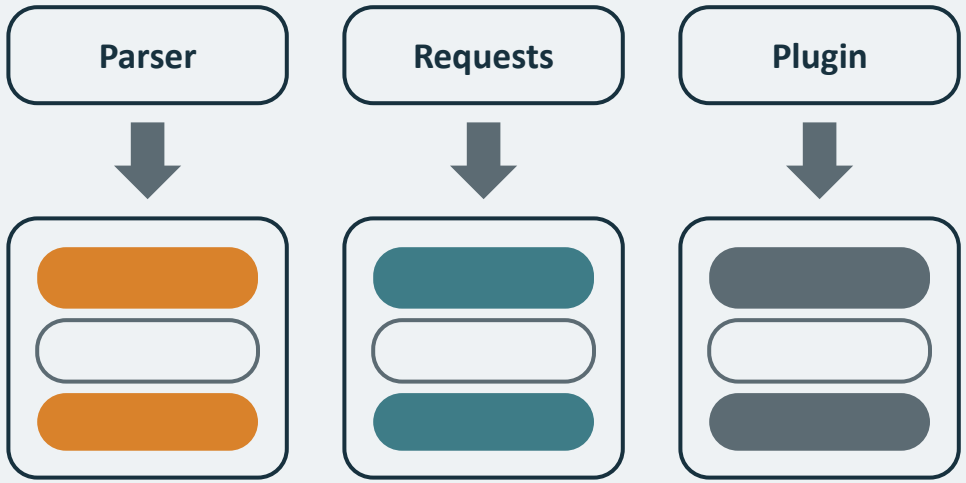
CookMem 1.6 · github.com/coconut2015/cookmem

A memory context is an allocator you can create

malloc and free: one shared heap



CookMem: a context for each job



1. **Works like malloc**
   allocate, deallocate and reallocate, one object at a time.

2. **Belongs to one job**
   Give a context to a parser, a request or a plugin.

3. **Easy to control**
   Cap it, check it, or throw the whole thing away.

Who CookMem is for

1

Complex pointer graphs

Compilers, parsers, database engines: free the whole graph at once.

2

Single-threaded performance

A context per thread: no locks, data kept close together.

3

Debugging without Valgrind

Padding checks, double frees and leak reports in normal builds.

4

Limiting memory

Footprint limits, also for libraries with allocation hooks.

Not a drop-in malloc replacement

jemalloc and mimalloc replace the allocator of a whole program.

With CookMem, you allocate from a context explicitly, one thread at a time.

What a context gives you

malloc-style API

allocate, deallocate, reallocate, allocateAligned

Everyone

Release everything

releaseAll () or destroying the context

Pointer graphs

Checks

Padding, double frees, and leak reports

Debugging

Limits and statistics

setFootprintLimit (), getFootprint (), StatsLogger

Limiting memory

Standard containers

Allocator<T> (C++11), MemResource (C++17)

Everyone

Your memory source

mmap or VirtualAlloc, malloc, your buffer

Performance

How the pieces fit together

Your code	containers, parsers, request handlers
Allocator<T> · MemResource	STL and std::pmr containers
MemContext	allocate, deallocate, releaseAll
MemPool	dlmalloc bins, best fit, merging free chunks
Arena	segments from mmap, malloc or your buffer

Combined with templates

No virtual calls between the layers, so the compiler can inline them.

Mix and match

Pick an arena and a logger; several contexts can share one arena.

Add CookMem to your project

```
include (FetchContent)
FetchContent_Declare (cookmem
    GIT_REPOSITORY https://github.com/coconut2015/cookmem.git
    GIT_TAG        v1.6.0)
FetchContent_MakeAvailable (cookmem)

target_link_libraries (myapp PRIVATE cookmem::cookmem)
```

CMakeLists.txt

```
#include <cookmem.h>
```

Then, in your code

1. **Header-only**
   Or add include/ to your include path.

2. **C++11**
   Only MemResource needs C++17.

3. **Installed copy**
   find_package (cookmem 1.6) after cmake --install.

Allocate, resize and free

```
cookmem::SimpleMemContext<> memCtx;

// Allocate, like malloc.
const char hello[] = "Hello, CookMem";
char* text = static_cast<char*> (memCtx.allocate (32));
std::memcpy (text, hello, sizeof (hello));
std::cout << text << "\n";

// Grow it, like realloc.  The contents are kept.
text = static_cast<char*> (memCtx.reallocate (text, 64));
std::cout << memCtx.getSize (text) << " bytes\n";

// Free it, like free.
memCtx.deallocate (text);
```

examples/01_first_allocation.cpp (shortened)

1. **allocate ()**
   Like malloc, with the same alignment.

2. **reallocate ()**
   Like realloc; grows in place when it can.

3. **deallocate ()**
   Like free; the memory is reused.

```
Hello, CookMem
64 bytes
```

Free everything at once

```
cookmem::SimpleMemContext<> memCtx;

for (int i = 0; i < 100000; ++i)
{
    memCtx.allocate (48);    // never freed one by one
}
std::cout << memCtx.getFootprint () << std::endl;

memCtx.releaseAll ();        // every pointer is now invalid
std::cout << memCtx.getFootprint () << std::endl;
```

6.4 MB

→ 0

footprint before and after one releaseAll ()

examples/02_release_all.cpp (shortened)

Use it for data that lives and dies together: everything allocated for one request, one frame or one parse.

No destructors run, so RAII does not happen.

Release everything at once only for objects that own nothing but memory from the same context. Destroy objects holding files, locks or global-heap memory first.

Standard containers

`cookmem::Allocator`

C++11

```
cookmem::CachedMemContext<> memCtx;
cookmem::Allocator<int> allocator (memCtx);

std::vector<int, cookmem::Allocator<int>>
    numbers (allocator);
numbers.push_back (42);
```

based on examples/03_stl_allocator.cpp

Any container, from C++11.

The allocator is part of the type.

Converts between element types.

`cookmem::MemResource`

C++17

```
cookmem::SimpleMemContext<> memCtx;
cookmem::MemResource<cookmem::SimpleMemContext<>>
    resource (memCtx);

std::pmr::map<int, std::pmr::string>
    names (&resource);
names[1] = "nested strings share it";
```

based on examples/04_pmr.cpp

One container type for any resource.

Nested containers use it automatically.

Throws std::bad_alloc when it is full.

Either way: destroy the containers before the context.

Checks and limits

Padding

Extra bytes after each allocation, checked when freed.

```
std::memset (buffer, 'x', 11);   // 10 allocated
memCtx.deallocate (buffer);
```

throws cookmem::Exception

Footprint limit

Caps what the context takes from the system.

```
memCtx.setFootprintLimit (1024 * 1024);
memCtx.allocate (10 * 1024 * 1024);
```

returns nullptr

Leak report

Lists the allocations never freed.

```
memCtx.forEachAllocation (
    [] (void*, std::size_t size) { /* ... */ });
```

```
Not freed: 100 bytes
```

examples/05_checks_and_limits.cpp (excerpts)

Choose a context



`SimpleMemContext<>`

Segments from mmap or VirtualAlloc, returned to the system when free.

```
SimpleMemContext<> memCtx;
```

General use



`CachedMemContext<>`

Keeps released segments and reuses them for later contexts.

```
CachedMemContext<> memCtx;
```

Contexts created often



`MemContext<FixedArena>`

Uses a buffer you provide; allocations fail when it is full.

```
FixedArena arena (buf, n);
```

No system calls

Three rules to remember

1

One thread per context

Contexts have no locks. Use each one from one thread at a time, typically one per thread.

2

Free with the same context

A pointer goes back to the context that allocated it. contains () can check where a pointer belongs.

3

Objects before contexts

Destroy containers and objects that own other resources before the context; releaseAll () calls no destructors.

Next steps

1. **Run the examples**
   `cmake -S . -B build && ctest --test-dir build`

2. **Read the documentation**
   coconut2015.github.io/cookmem

3. **Coming next**
   Tutorials 2 to 5: pointer graphs, performance, debugging, memory limits

